

Optimal Reward Allocation via Proportional Splitting

Anonymous
Anonymous

Abstract—Following the publication of Bitcoin’s arguably most famous attack, selfish mining, various works have introduced mechanisms to enhance blockchain systems’ game theoretic resilience. Some reward mechanisms, like FruitChains, have been shown to be equilibria in theory. However, their guarantees assume non-realistic parameters and their performance degrades significantly in a practical deployment setting. In this work we introduce a reward allocation mechanism, called Proportional Splitting (PRS), which outperforms existing state of the art. We show that, for large enough parameters, PRS is an equilibrium, offering the same theoretical guarantees as the state of the art. In addition, for practical, realistically small, parameters, PRS outperforms all existing reward mechanisms across an array of metrics. We implement PRS on top of a variant of PoEM, a Proof-of-Work (PoW) protocol that enables a more accurate estimation of each party’s mining power compared to e.g., Bitcoin. We then evaluate PRS both theoretically and in practice. On the theoretical side, we show that our protocol combined with PRS is an equilibrium and guarantees fairness, similar to FruitChains. In practice, we compare PRS with an array of existing reward mechanisms and show that, assuming an accurate estimation of the mining power distribution, it outperforms them across various well-established metrics. Finally, we realize this assumption by approximating the power distribution via low-work objects called “workshares” and quantify the tradeoff between the approximation’s accuracy and storage overhead.

I. INTRODUCTION

Bitcoin [1] enabled, for the first time, the creation of a fully open and decentralized system. Its first application, the Bitcoin cryptocurrency, gave rise to a swathe of distributed financial applications. As Bitcoin gained traction though, it was more closely and rigorously analyzed.

The first major and, arguably, most famous attack against Bitcoin was proposed only 3 years after its launch and bore the name *selfish mining* [2]. The novel element of this attack was its game theoretic analysis, which employed a previously unexplored utility function, namely proportional (instead of absolute) rewards, and showed that Bitcoin is not an equilibrium under this model. Following, various works built on the idea of selfish mining and optimized it [3], [4], [5].

An elegant result that addressed selfish mining was FruitChains [6]. This protocol satisfies *fairness*, meaning that rewards are distributed approximately proportionally to each

participant’s power. The main idea of FruitChains was to use objects that satisfy a level of Proof-of-Work (PoW) that is lower than the level required for block generation. These objects, called “fruits”, are created more frequently and allow a more accurate estimation of the distribution of mining power among parties. Although FruitChains was shown — in theory — to be an equilibrium, its guarantees were satisfied for unrealistically large parameter values.¹ Intuitively, guaranteeing fairness required a finality parameter in the order of a few days, which is impractical for day-to-day transactions that need to be finalized as soon as possible.

Following, a detailed evaluation of various reward distribution mechanisms, namely Nakamoto Consensus [1], FruitChains [6], Reward Splitting (RS) [7], [8], [9], and Subchains [10] was performed in [7]. Crucially, this work established a framework of common metrics, with respect to which the different mechanisms were evaluated and compared. These metrics include (i) incentive compatibility, that is the level of fairness in reward distribution, (ii) subversion gain, i.e., the profitability of double-spending attacks, and (iii) censorship susceptibility, that is the income loss that honest parties may incur due to an attack. Interestingly, for realistic and practical parameter values, FruitChains underperformed some of the alternative reward distribution mechanisms w.r.t. most of the analyzed metrics.

At this point, a pertinent question is creating a reward mechanism which both achieves optimal theoretical guarantees and performs optimally in a realistic setting. Specifically, the reward mechanism should be an equilibrium and guarantee fairness, like FruitChains, while also outperforming the other reward mechanisms when using practical finality parameters, e.g., 10-block confirmation windows.² Our work sets out to answer this question affirmatively, by presenting a mechanism called *Proportional Reward Splitting (PRS)*.

Proportional Reward Splitting (PRS) borrows ideas from other mechanisms, namely FruitChains and Reward Splitting (RS), and combines them to produce a qualitatively better result. PRS is similar to RS in that it distributes rewards among all objects of the same height. However, instead of only counting blocks, PRS also counts objects of lower amount PoW, called “workshares”. This idea is borrowed from FruitChains, where lower-PoW objects (“fruits”) enable a more accurate

¹FruitChains defines a window, within which fruits need to be published on-chain, of length $R\kappa$, where R is a “recency” parameter with proposed value 17 and κ is the security parameter.

²10 blocks is traditionally the confirmation window used for Bitcoin.

distribution of reward proportionally to parties' mining power. Therefore, if the adversary controls 30% of mining power, then for each height it receives approx. 30% of the height's reward.

As shown in our theoretical analysis and practical evaluation, the PRS mechanism combines the best of both worlds, by achieving the same theoretical guarantees as FruitChains (namely being an equilibrium and guaranteeing fairness with large enough parameters) and outperforming all other mechanisms in a realistic setting w.r.t. the metrics outlined in [7].

In practice, our evaluation shows that PRS both lowers an adversary's expected benefits and increases the threshold of mining power, after which an adversary can gain more benefits than running the honest protocol. For example, when it comes to the proportional share of rewards that an adversary A gains (incentive compatibility), using PRS requires A to have more than 38% of mining power to get disproportionately more rewards, compared to the threshold of 35% of the second-best option (RS). Additionally, an adversary with 38% of mining power would receive 38% of all rewards, compared to 40% under RS and more than 50% under Bitcoin or FruitChains. Therefore, PRS both increases a system's resilience and reduces the expected gains of deviations.

Our Contributions and Roadmap

Section III describes the protocol (PoEM²) run by the miners. PoEM² is a variant of PoEM [11], in that it uses the same fork choice rule and block generation process, but additionally records (on-chain) objects of lower PoW than blocks, called "workshares". These objects are generated in the main loop of the protocol and have the same structure as blocks, albeit satisfy a lower PoW threshold. On each round, every party propagates the newly-created workshares and attempts to publish all unpublished workshares in a dedicated structure within the block. The recording of workshares is the only difference of PoEM² compared to PoEM, therefore it directly inherits PoEM's security guarantees while also enabling a more accurate estimation of each party's mining power.

Section IV defines the PRS allocation mechanism. This mechanism uses workshares to split each block's rewards proportionally to each party's mining power. Since the real mining power distribution is not known, workshares enable only an approximate estimation.

This consideration is explored first in Section V, which outlines the theoretical evaluation of PRS, proving that, for large enough parameters, our proposed protocol is an equilibrium guaranteeing approximate fairness.

Next, Section VI evaluates the performance of PRS w.r.t. alternative mechanisms, namely Nakamoto Consensus, FruitChains, and Reward Splitting, in a practical setting. In particular, we evaluate the mechanisms by constructing a Markov Decision Process (MDP) for each one and defining a reward function that implements the analyzed metrics. In essence, the MDP identifies the strategy that maximizes utility in a stochastic environment. Importantly, we parameterize each MDP with realistic values, e.g., regarding block finality, in order to evaluate how each mechanism is expected to perform

in practice. Our evaluation shows that PRS outperforms all other mechanisms w.r.t. all metrics outlined in [7], as long as an accurate estimation of the power distribution is available. To this end, the final part of Section VI discusses how an accurate approximation of the power distribution can be performed using workshares and outlines the tradeoff between accuracy and storage overhead, that is the amount of extra data that need to be published on-chain.

Related Work

The first point of comparison is FruitChains [6]. FruitChains was shown to be a ρ -coalition-safe ϵ -Nash equilibrium, i.e., any coalition of parties with fraction of power less than ρ cannot gain more than ϵ of their fair share of rewards. As we will show in Section V, our mechanism achieves the same guarantee, as the analysis of FruitChains carries over in our case almost directly. Nonetheless, PRS outperforms FruitChains in practice, that is for parameter values smaller than the theoretical, unrealistic, bounds (cf. Section VI).

Recording low-work objects on-chain was explored in StrongChain [12]. StrongChain publishes workshares on-chain, as opposed to keeping them within a mining pool and invisible to the protocol. The paper offers a protocol implementation and a brief discussion of the protocol's performance in the optimistic setting, where all parties are honest. However, StrongChain analysis is lacking in many aspects. Specifically, StrongChain offers only a high level description of reward allocation and an intuition of why this allocation is beneficial. In comparison, our mechanism is analyzed both in theory, in a game theoretic setting where parties are assumed rational instead of simply honest, and in practice via Markov decision processes (MDP). Additionally, StrongChain's analysis considers only a specific instance of a selfish mining strategy, and analyzes the protocol's performance under various parameter values, whereas our MDP analysis evaluates the performance of the optimal adversarial strategy.

Another line of work on recording low-work objects on-chain builds on the idea of "weak blocks". These works include Subchains [10] and Flux [13]. Here, the main idea is the formation of "sub-chains", that is chains of objects with less amount of PoW than blocks. Therefore, between two consecutive (heavy) blocks, a sub-chain of weak blocks may be created, enabling a more fine-grained estimation of the power distribution among participants. Flux offered a simulation-based analysis, showing that it performs better w.r.t. selfish mining compared to Bitcoin. However, as was shown in [7], Subchains performs worse in practice compared to Reward Splitting (RS). In comparison, our mechanism outperforms RS, and thus also Subchains, under the framework of [7].

Finally, Section III, which describes the mining protocol, makes use of the PoEM [11] fork choice rule. Our description changes the original mining algorithm only regarding the creation and publishing of workshare objects, so the security properties of PoEM are inherited directly in our case. In this work, we chose PoEM due to its definition of inherent work, which offers a better approximation of the

power distribution among participants. Nonetheless, any chain-based fork-choice rule, such as Nakamoto Consensus [1], uniform tie-breaking [14], [2], smallest hash tie-breaking [8], unpredictable deterministic tie-breaking [15], or Publish-or-Perish [16], would suffice, as long as the distributed ledger protocol that implements it guarantees safety and liveness.

II. PRELIMINARIES

We now provide the necessary background on which our protocol and analysis are built. We first define the notation that will be used throughout our work. Next, we outline the cryptographic and game theoretic models of our analysis. Finally, we briefly describe the fork-choice rule protocol PoEM [11], which will be the basis on top of which our reward mechanism will be implemented.

A. Notation and Cryptographic Primitives

In the rest of the paper we will use the following notation:

- $a.b$: the element b of a complex object a
- $A \parallel A'$: the concatenation of A and A' , which might be blocks, chains, or (ledgers of) transactions
- $C[i]$: the i -th block of a chain C ; if $i = -1$ then it denotes the last block (also called head or tip)
- $C[:i], C[-i:]$: the first (resp. last) i blocks of a chain C
- $L \prec L'$: the prefix operation, i.e., L is a prefix of L'
- B_{tx} : the block B in which a transaction tx is published
- $|\cdot|$: the length of the given element, that is if the element is a list then the number of items in the list, if it is a bitstring then the number of bits, etc
- *view*: short hand for execution trace view
- w : wait time (liveness) parameter
- k : common prefix parameter
- Δ : network delay
- p : probability that an honest block is mined in a round
- ρ : adversary mining power

We will also use the following primitives:

- $H(\cdot)$: a hash function that, given an arbitrary bitstring, outputs a value of λ bits³ and models a random oracle.
- $d(\cdot)$: a digest function that, given a list of arbitrary bitstrings, outputs a value of λ bits⁴

B. Execution Model

Our analysis assumes a multi-party setting, following the model of Bitcoin Backbone [17]. An “environment” program $\mathcal{Z}$ drives the execution of a protocol Π , by spawning for each party $\mathcal{P}$ an instance of an “interactive Turing machine” (ITM) which executes the protocol. Interaction between parties is controlled by a program C , s.t. $(\mathcal{Z}, C)$ create a system of ITMs. Our analysis applies to “locally polynomially bounded” systems, ensuring polynomial time execution.

Network. The execution proceeds in (a polynomial number of) rounds of size Δ . In each round r , each party is activated

and performs various operations based on its algorithm. An honest message produced at round r is delivered to all other parties at the beginning of round $r + 1$, i.e., our analysis assumes a *synchronous network*. Finally, we assume a diffuse functionality, i.e., a gossip protocol which allows the parties to share messages without the need of a fully connected graph.

Parties. The set of n parties $\mathbb{P}$ remains fixed for the duration of the execution. Each party $\mathcal{P}$ can make a fixed number of queries to a hashing oracle, which enables participation in the distributed ledger protocol. The adversary controls at most $t = \rho \cdot n$ parties, out of the total n at any given time, and its power is the sum of the corrupted parties’ power.

Proof-of-Work. All parties have access to a random oracle $\mathcal{O}_{\text{rnd}}$. Each party can perform q of queries to the oracle on each round, while the adversary can perform $t \cdot q$ queries per round. Additionally, the protocol defines a *difficulty parameter* p , which denotes the probability that a single query to $\mathcal{O}_{\text{rnd}}$ results in the successful creation of a block. In practice, the random oracle is instantiated as a hash function $H(\cdot)$. We note that the codomain of H is large enough, s.t. when a party makes a batch of queries, the above probability is practically independent for each query.

C. Cryptographic Model

In the cryptographic model, some parties are assumed honest (i.e., they follow the protocol) whereas the rest are Byzantine (i.e., they act arbitrarily). Note that incentives do not play a role in the security analysis under this model, and instead will be considered in the game theoretic analysis.

Adversary. The adversary A is an ITM which, upon activation, may “corrupt” a number of parties, by sending a relevant message to the controller after $\mathcal{Z}$ instructs it to do so. Following, when a corrupted party is supposed to be activated, A is activated instead. Furthermore, A is “adaptive”, i.e., corrupts parties on the fly, and “rushing”, i.e., decides its strategy and acts after observing the other parties’ messages.

Randomized Execution. The execution of a protocol is probabilistic, with randomness coming from both honest parties and the adversary, denoted as A , which controls all corrupted nodes, as well as the environment $\mathcal{Z}$, which provides inputs to honest nodes throughout the protocol’s execution.

We denote a randomly sampled execution trace as $\text{view} \xleftarrow{\$} \text{EXEC}^{\Pi}(A, \mathcal{Z}, \kappa)$, where $|\text{view}|$ represents the number of rounds in the execution trace. Specifically, *view* is a random variable that captures the joint perspective of all parties, encompassing their inputs, random coins, and received messages (including those from the random oracle). This joint *view* uniquely determines the entire execution.

Restrictions on the Environment. Constraints on $(A, \mathcal{Z})$ [17]. The environment $\mathcal{Z}$ and the adversary A must respect certain constraints. We say that a p.p.t. pair $(A, \mathcal{Z})$ is (n, ρ, Δ) -respecting w.r.t. Π , iff for every $\kappa \in \mathbb{N}$, every *view* in the support of $\text{EXEC}^{\Pi}(A, \mathcal{Z}, \kappa)$, the following holds:

- $\mathcal{Z}$ activates n parties in *view*;

³ H should satisfy the standard hash function properties, namely collision, preimage, and second-preimage resistance.

⁴ d should also satisfy collision resistance and can be implemented e.g., via a Merkle tree, where $d(\cdot)$ returns the tree’s root.

- For any message broadcast by an honest player at any time t in view, any player that is honest at time $t + \Delta$ or later must have received the message. This means that in the case of newly spawned players, instantly delivers messages that were sent more than Δ rounds ago. As long as this Δ constraint is respected, A is allowed to delay or reorder honest players' messages arbitrarily.
- At every round r in view, A controls at most $\rho \cdot n$ parties.

Let $\Gamma(\cdot, \cdot, \cdot)$ be a boolean predicate. We say that a p.p.t. pair (A, Z) is Γ -compliant w.r.t. protocol π iff: (i) (A, Z) is (n, ρ, Δ) -respecting w.r.t. π ; (ii) $\Gamma(n, \rho, \Delta) = 1$.

D. Game Theoretic Model

In the game theoretic model we assume that all parties are rational. Specifically, each party $\mathcal{P}$ employs a strategy S , that is a set of rules and actions that the party makes depending on its input. In other words, S defines the part of the distributed protocol that $\mathcal{P}$ performs.

Each party has a well-defined utility U and chooses a strategy S with the goal of maximizing their utility. The literature of game theoretic analyses of blockchain protocols considers various types of utilities [18]: (i) absolute rewards; (ii) absolute profit, i.e., absolute rewards minus cost; (iii) relative rewards, i.e., the percentage of the party's reward among all allocated rewards; (iv) relative profit, i.e., relative rewards minus (absolute) cost; (v) deposits and penalties, e.g., using a slashing mechanism; (vi) external rewards, e.g., bribes.

A strategy profile σ is a vector of all parties' strategies and it is an ϵ -Nash equilibrium if no party can increase its utility more than ϵ by unilaterally changing its strategy.

E. Property Definitions

As our analysis is based on [17] and [19], we first recall the definitions of chain growth, chain quality, and consistency from there. Following, we revisit the fairness definition of [6].

1) Chain Growth:

$$\text{min-chain-increase}_{t,t'}(\text{view}) = \min_{i,j} |\text{chain}_j^{t+t'}(\text{view})| - |\text{chain}_i^t(\text{view})|$$

$$\text{max-chain-increase}_{t,t'}(\text{view}) = \max_{i,j} |\text{chain}_j^{t+t'}(\text{view})| - |\text{chain}_i^t(\text{view})|$$

where we quantify over nodes i, j such that i is honest at round t and j is honest at round $t + t'$ in view.

Let $\text{growth}_{t_0, t_1}^T(\text{view}, \Delta, T) = 1$ iff the following hold:

- **(consistent length)** for all time steps $t \leq |\text{view}| - \Delta$, $t + \Delta \leq t' \leq |\text{view}|$, for every two players i, j such that in view i is honest at t and j is honest at t' , we have that $|\text{chain}_j^{t'}(\text{view})| \geq |\text{chain}_i^t(\text{view})|$
- **(chain growth lower bound)** for every time step $t \leq |\text{view}| - t_0$, we have $\text{min-chain-increase}_{t, t_0}(\text{view}) \geq T$.
- **(chain growth upper bound)** for every time step $t \leq |\text{view}| - t_1$, we have $\text{max-chain-increase}_{t, t_1}(\text{view}) \leq T$.

Definition 1 (Chain growth). A blockchain protocol Π satisfies (T_0, g_0, g_1) -chain growth in Γ -environments, if for all Γ -compliant p.p.t. pair (A, Z) , there exists some negligible

function negl such that for every $\kappa \in \mathbb{N}$, $T \geq T_0$, $t_0 \geq \frac{T}{g_0}$ and $t_1 \leq \frac{T}{g_1}$ the following holds:

$$\Pr[\text{view} \leftarrow \text{EXEC}^\Pi(A, Z, \kappa) : \text{growth}_{t_0, t_1}^T(\text{view}, \Delta, \kappa) = 1] \geq 1 - \text{negl}(\kappa)$$

2) **Chain Quality:** The second desired property is that the number of blocks contributed by the adversary is not too large. Given a chain, we say that a block $B := \text{chain}[j]$ is honest w.r.t. view and prefix $\text{chain}[:j']$ where $j' < j$ if in view there exists some node i honest at some time $t \leq |\text{view}|$, such that (i) $\text{chain}[:j'] \prec C_i^t(\text{view})$ where $\prec$ denotes "is a prefix of" and (ii) Z input B to node i at time t . Informally, for an honest node's chain, a block $B := \text{chain}[j]$ is honest w.r.t. a prefix $\text{chain}[:j']$ where $j' < j$, if earlier there is some honest node who received B as input when its local chain contains the prefix $\text{chain}[:j']$.

Let $\text{quality}^T(\text{view}, \mu) = 1$ iff for every time t and every player i such that i is honest at t in view, among any consecutive sequence of T blocks $\text{chain}[j + 1 \dots j + T] \subseteq \text{chain}_i^t(\text{view})$, the fraction of blocks that are honest w.r.t. view and $\text{chain}[:j]$ is at least μ .

Definition 2 (Chain quality). A blockchain protocol Π has (T_0, μ) -chain quality, in Γ -environments if for all Γ -compliant p.p.t. pair (A, Z) , there exists some negligible function negl such that for every $\kappa \in \mathbb{N}$ and every $T \geq T_0$ the following holds:

$$\Pr[\text{view} \leftarrow \text{EXEC}^\Pi(A, Z, \kappa) : \text{quality}^T(\text{view}, \mu) = 1] \geq 1 - \text{negl}(\kappa)$$

3) **Consistency:** Let $\text{consistent}^T(\text{view}) = 1$ iff for all times $t \leq t_0$, and all players i, j (potentially the same) such that i is honest at t and j is honest at t' in view, we have that the prefixes of $\text{chain}_i^t(\text{view})$ and $\text{chain}_j^{t'}(\text{view})$ consisting of the first $\ell = |\text{chain}_i^t(\text{view})| - T$ records are identical — this also implies that the following must be true: $|\text{chain}_j^{t'}(\text{view})| > \ell$, i.e., $\text{chain}_j^{t'}(\text{view})$ cannot be too much shorter than $\text{chain}_i^t(\text{view})$ given that $t' \geq t$.

Definition 3 (Consistency). A blockchain protocol Π satisfies T_0 -consistency, in Γ -environments if for all Γ -compliant p.p.t. pair (A, Z) , there exists some negligible function negl such that for every $\kappa \in \mathbb{N}$ and every $T \geq T_0$ the following holds:

$$\Pr[\text{view} \leftarrow \text{EXEC}^\Pi(A, Z, \kappa) : \text{consistent}^T(\text{view}) = 1] \geq 1 - \text{negl}(\kappa)$$

4) **Fairness:** The results from [19] are parameterized by the following quantities:

- $\alpha := 1 - (1 - p)^{(1-\rho)n}$: the probability that some honest player succeeds in mining a block in a round.
- $\beta := \rho np$: the expected number of blocks an attacker can mine in a round.
- $\gamma := \frac{\alpha}{1 + \Delta \alpha}$: a discounted version of α which takes into account that messages sent by honest parties can be

delayed by up to Δ rounds and this can lead to honest players redoing work; in essence, γ corresponds to the effective mining power.

Compliant executions for Nakamoto’s blockchain. We now specify the compliance predicate Γ_{nak}^p for Nakamoto’s blockchain. We say that $\Gamma_{\text{nak}}^p = 1$ iff there is a constant $\lambda > 1$, such that

$$\alpha(1 - 2(\Delta + 1)\alpha) \geq \lambda\beta$$

where α and β are functions of the parameters n, ρ, Δ and κ as defined above. As shown in [19], this condition implies the following:

Fact 1. *If (A, Z) is a Γ_{nak}^p -compliant, then $np\Delta < 1$.*

Fact 2. *If (A, Z) is a Γ_{nak}^p -compliant, then $\gamma \geq \frac{np}{8}$.*

Following, FruitChains [6] provided a definition for fairness as follows.

Let a player subset selection, $S(\text{view}, r)$, be a function that given (view, r) outputs a subset of the players that are honest at round r in view. S is a ϕ -fraction player subset selection if $S(\text{view}, r)$ always outputs a set of size ϕn (rounded upwards) where n is the number of players in view.

Given a player subset selection S , a *record* m is *S-compatible w.r.t. view and prefix chain* if there exists a player j and round r' such that j is in $S(\text{view}, r_0)$, the environment provided m as an input to j at round r' , and chain $\prec \text{chain}_{r'}^T(\text{view})$, where $\prec$ denotes “is a prefix of”.

Let $\text{quality}_{T,S}^T(\text{view}, \mu) = 1$ iff for every round r and every player i such that i is honest in round r of view, we have that among any consecutive sequence of T records $\text{chain}_i^T(\text{view})[j+1 : j+T]$, the fraction of records that are *S-compatible w.r.t. view and prefix* $\text{chain}_i^T(\text{view})[: j]$ is at least μ .

Definition 4 (Fairness). *A blockchain protocol Π has (approximate) fairness (T_0, δ) in Γ -environments, if for all Γ -compliant p.p.t. (A, Z) , every positive constant $\phi \leq 1 - \rho$, every ϕ -fraction subset selection S , there exists some negligible function ϵ such that for every $\kappa \in \mathbb{N}$ and every $T \geq T_0$ the following holds:*

$$\Pr[\text{view} \leftarrow \text{EXEC}^\Pi(A, Z, \kappa) : \text{quality}_{T,S}^T(\text{view}, (1 - \delta)\phi) = 1] \geq 1 - \epsilon(\kappa)$$

III. PoEM² PROTOCOL

In this section we introduce a variant of PoEM [11], a Proof-of-Work (PoW) consensus protocol.

The major contribution of PoEM is the introduction of the notion of an object’s *intrinsic work*. Given an object obj , its intrinsic work is computed as:

$$\text{WORK}(\text{obj}) = \lambda - \lfloor \log H(\text{obj}) \rfloor \quad (1)$$

In essence, the intrinsic work is equivalent to the number of consecutive most significant zero bits in the object’s hash: the more zero bits, the higher the intrinsic work.⁵

⁵ λ is the security parameter.

Intrinsic work is useful in our case, since it gives a more accurate estimation of the miners’ power compared to using blocks themselves, as in Bitcoin. In PoEM² we enhance this estimation by introducing low-work objects (“workshares”). Workshares have the same structure as blocks and are created in the same (main) protocol loop, albeit satisfy a lower work threshold. Workshares are propagated through the network and published in blocks similarly to transactions. Since workshares and blocks are the output of the same function, miners do not choose between creating either of the two. Additionally, miners don’t know *a priori* whether a loop execution will produce a workshare or block (or neither), since the hash function in our work models a random oracle.

The generation and on-chain recording of workshares is the only difference between PoEM and PoEM². Therefore, since workshares do not affect the fork choice rule or the block generation process, PoEM² directly inherits the security properties of PoEM. Following, we first describe the data objects in our protocol, namely blocks and workshares. Next, we describe the core protocol algorithm, along with the chain selection and the ledger extraction mechanisms.

A. Blocks and Workshares

A work object in PoEM² has the following format:

$$\text{WorkObj} = \langle (d(\text{TX}); d(\text{WS}); h_{B'}; h_{\hat{B}}), \eta \rangle$$

where:

- TX: a list of transactions that are published in WorkObj (when WorkObj is treated as a block);
- WS: a list of Workobjects, which can be workshares or uncles,⁶ that are published in WorkObj (when WorkObj is treated as a block);
- $h_{B'}$: the reference to a block B' , i.e., $H(B')$;
- $h_{\hat{B}}$: the reference to a block $\hat{B}$, i.e., $H(\hat{B})$;
- η : a nonce created during the PoW execution.

Note: The work object has the same structure as the block’s header. In particular, a full block is created by concatenating the work object (that is, its header) with the lists of transactions TX and workshares WS.

A work object can be a workshare or a block, depending on its intrinsic work. When an object is interpreted as a workshare, the elements $d(\text{TX}), d(\text{WS})$ are ignored. Instead, these elements are only used for the chain selection and ledger extraction mechanisms, which only parse a chain’s blocks.

In more detail, the validity rules for blocks and workshares are as follows. We also assume the existence of a transaction validity predicate, which specifies the application logic of the ledger, e.g., preventing double spending.

Block validity. $B = \langle (d(\text{TX}); d(\text{WS}); h_{B'}; h_{\hat{B}}), \eta \rangle, \text{TX}$ is valid w.r.t. a chain C if the following conditions hold:

- $d(\text{TX})$ is the digest of TX.
- $h_{B'}, h_{\hat{B}}$ are references to $C[-1], C[-k]$ respectively.

⁶An uncle satisfies the work threshold of a block, but is not part of the canonical chain, i.e., it has been “forked out”.

- Every Workobject (workshare or uncle) in WS is valid w.r.t. C (see below);
- No Workobject is present in C multiple times.
- Every transaction in TX satisfies the protocol's validity predicate.
- $\text{WORK}(B) > T_{\text{block}}$, where T_{block} is a protocol parameter that defines the PoW lower bound for a block to be valid.

Uncle validity. $B = \langle (d(\text{TX}); d(\text{WS}); h_{B'}; h_{\bar{B}}), \eta \rangle$, TX) is a valid uncle w.r.t. chain C , if all of the following hold:

- B is included in WS of exactly one block $\bar{B}$ in C and $B \notin C$;
- $h_{B'}$ is a reference to $C'[-1]$, which in turn is either a valid block in C or a valid uncle w.r.t. C ;
- $h_{\bar{B}}$ is a reference to $C'[-k]$, which is a valid block in C ;
- Let $\bar{C}$ be the prefix of C that goes up to this block $\bar{B}$. $|C'|$ is between $|\bar{C}| - W_{\text{recent}}$ and $|\bar{C}| - 1$;
- Following the reference $h_{B'}$ to a block B' , and then this blocks' reference and so on at most k times, gives a block that is part of C ;
- $\text{WORK}(B) > T_{\text{block}}$.

Workshare validity. $\text{ws} = \langle (d(\text{TX}); d(\text{WS}); h_{B'}; h_{\bar{B}}), \eta \rangle$ is valid w.r.t. chain C if:

- ws is included in WS of exactly one block $\bar{B}$ in C ;
- Let $\bar{C}$ be the prefix of C that goes up to this block $\bar{B}$. $h_{B'}$ is a reference to any valid block or uncle with a height between $|\bar{C}| - W_{\text{recent}}$ and $|\bar{C}| - 1$;
- $\text{WORK}(\text{ws}) > T_{\text{ws}}$, where T_{ws} is a protocol parameter that defines the PoW lower bound for a workshare to be valid.

Intuitively, T_{ws} and T_{block} define the minimum amount of intrinsic work that a workshare or block should have respectively and it holds that $T_{\text{ws}} \leq T_{\text{block}}$.

Chain validity. Finally, a chain C is valid w.r.t. a genesis block G (denoted $\text{VALIDATE}_G(C)$) if every block in C is valid and they form a chain that starts from G .

B. Main Protocol

The main function of PoEM^2 is defined in Algorithm 1. First, the party is constructed using the **CONSTRUCTOR** function (Line 3). In every round, each party is executed by the environment using function **EXECUTE** (this function is due to the round-based nature of our time model). The algorithm maintains a local chain C , which is initialized to the genesis block G , and proceeds as follows.

Chains. The party first receives from the network the set of available chains via the function **RECEIVE** (Line 8). Out of all the available chains, including the local chain C , the heaviest chain is picked by applying the **MAXVALID** function (described in [11]).

Mempool. The party receives the set of transactions in the mempool TX via the function **INPUT** (Line 10) and attempts to publish them in the newly-mined block. Specifically, the party orders the transactions in TX in an arbitrary manner in order to form the list TX; for ease of notation, we abstract the ordering

Algorithm 1 The PoEM^2 protocol

```

1:  $G \leftarrow \emptyset$ 
2:  $C \leftarrow []$ 
3: function CONSTRUCTOR( $G'$ )
4:    $G \leftarrow G'$  ▷ Select Genesis Block
5:    $C \leftarrow [G]$  ▷ Add Genesis Block to start of chain
6:   round  $\leftarrow 1$ 
7: function EXECUTE $_{d,H}(1^\lambda)$ 
8:    $\mathbb{C} \leftarrow \text{RECEIVE}()$  ▷ Receive chains from the network
9:    $C \leftarrow \text{MAXVALID}(\{C\} \cup \mathbb{C})$  ▷ Adopt heaviest chain
   (cf. [11])
10:   $\langle \text{TX}, \text{WS} \rangle \leftarrow \text{INPUT}()$ 
11:   $\mathbb{B} \leftarrow \emptyset$ 
12:   $\eta \xleftarrow{\$} \{0, 1\}^\lambda$ 
13:  for  $i \leftarrow 1$  to  $q$  do
14:     $\text{WorkObj} \leftarrow \langle (d(\text{TX}); d(\text{WS}); h_{C[-1]}; h_{C[-k]}), \eta \rangle$ 
15:    if  $\text{WORK}(\text{WorkObj}) > T_{\text{ws}}$  then
16:       $\text{WS} \leftarrow \text{WS} \parallel \text{WorkObj}$ 
17:    if  $\text{WORK}(\text{WorkObj}) > T_{\text{block}}$  then
18:       $B \leftarrow \langle \text{WorkObj}, \text{TX} \rangle$ 
19:       $\mathbb{B} \leftarrow \mathbb{B} \parallel B$ 
20:       $C \leftarrow C \parallel B$ 
21:     $\eta \leftarrow \eta + 1$ 
22:  DIFFUSE( $C, \text{WS} \parallel \mathbb{B}$ )
23:  round  $\leftarrow \text{round} + 1$ 

```

process outside of the protocol of Algorithm 1. Following, the list's digest $d(\text{TX})$ is included in the new block's header.

Workshares. The party receives from the network workshares WS, including uncle blocks (Line 10), and attempts to publish them in the new block. As opposed to transactions, the order of workshares does not matter, since they are only used for reward allocation (cf. Section IV).

Note: TX and WS are both sanitized to contain valid objects w.r.t. the party's local chain C , so that the transactions and workshares can be published in a new block. For ease of notation we omit this process from Algorithm 1 and assume that TX and WS (obtained from **INPUT**) are sanitized w.r.t. the local chain C at all rounds. In particular, all workshares fulfill the PoW inequality, there are not duplicate workshares in the chain, and uncle blocks are not part of the chain.

Proof-of-Work loop. The main part of the protocol is the PoW loop, during which the party *potentially* creates workshares and blocks. Specifically, on each round the party can execute q queries to the hash function. For each query, the work object includes a PoW nonce η and, depending on the amount of work that the object has, it may be a valid workshare and/or block. If a workshare is created, then it is immediately inserted to the list WS. This has the implication that a workshare which is created on some round r may be published on a block that is created on the same round r . Similarly, if a block is created, it is immediately appended to the local chain C . Regardless of whether a workshare or block is created, the party exhausts all

available hashing queries at their disposal without interrupting the PoW loop. Therefore, a party that follows the protocol may create multiple workshares and/or blocks on a single round.

Finally, the party diffuses to the network their local chain and list of workshares and proceeds to the next round.

C. Chain Selection and Ledger Extraction

Chain selection and ledger extraction are the same as PoEM [11]. For chain selection, the chain with the most intrinsic work (i.e., the sum of its blocks' intrinsic work $\text{WORK}(B)$) is chosen.

Algorithm 2 The PoEM maxvalid algorithm

```

1: function MAXVALIDG,T( $\bar{C}$ )
2:    $C_{\max} \leftarrow []$ 
3:    $\text{maxwork} \leftarrow 0$ 
4:   for  $C \in \bar{C}$  do
5:     if  $\neg \text{VALIDATE}_G(C)$  then
6:       continue
7:      $\text{thiswork} \leftarrow \widetilde{\text{WORK}}(C)$  ▷ Computed as
8:        $\sum_{B \in C} H(B) : \lambda$ 
9:     if  $\text{thiswork} > \text{maxwork}$  then
10:       $C_{\max} \leftarrow C$ 
11:       $\text{maxwork} \leftarrow \text{thiswork}$ 
12:   return  $C_{\max}$ 

```

Ledger extraction is parameterized by the safety parameter k . Given a chain C , the last k blocks are excluded from the ledger extraction process. The remaining blocks in C are parsed sequentially, starting from the genesis block G , and the ledger L is formed by concatenating each transaction in the list TX of each block.

Algorithm 3 The PoEM² ledger extraction mechanism

```

1: function EXTRACTk,G( $C$ )
2:   if  $\neg \text{VALIDATE}_G(C)$  then
3:     return  $\perp$ 
4:    $L \leftarrow []$  ▷ Initialize empty ledger
5:   for  $B \in C[-k:]$  do ▷ Parse each stable block
6:     for  $\text{tx} \in B.\text{TX}$  do
7:        $L \leftarrow L \parallel \text{tx.m}$ 
8:   return  $L$ 

```

IV. PROPORTIONAL REWARD SPLITTING

Our main contribution is the proposal of the proportional reward splitting reward mechanism.

Our mechanism builds on the idea of reward splitting [7], [8], [9]. Briefly, reward splitting defines a reward per height of the blockchain. The reward is split among all blocks that are at the same height, that is both the canonical block and “uncle” blocks, i.e., blocks that are part of a fork. Notably, this mechanism requires that uncle blocks are published on the main chain within a time window, after which the reward distribution of the given height is decided.

Proportional reward splitting is similar to reward splitting as it distributes rewards among all objects of the same height. However, it also has some core differences.

First, rewards are split between the honest parties and the adversary proportionally to each side's mining power. For example, if the adversary controls 30% of mining power, then for each height it receives 30% of the height's reward.⁷ Measurements for the mining power distribution are made on every slot, in order to capture fluctuations in the total mining power (and its distribution) almost as soon as they happen.

Second, we introduce another eligibility window, k , which concerns forks. Specifically, this window defines the depth of a fork up to which objects (that point to blocks of that depth) are eligible for rewards. For example, if an object (workshare or block) points to the fifth block of a fork, then it is eligible for rewards only if the fork eligibility window parameter k is at least 5. This is illustrated in Figure 1.

More specifically, the reward for a given height h is determined at height $h + R$. Every work object (block, workshare, uncle) of that height gets a reward according to the work relative to the total work of all objects at height h . Note that the height of a workshare or uncle is determined by the block it references $h_{B'}$, not the height of the block it is contained in. Further, to exclude uncle blocks that are building on invalid forks, we set k to the consistency parameter T_0 , achieved by the underlying protocol.

Note that delays in reward attribution are not uncommon in blockchain systems. For example, Bitcoin sets a 100 block requirement before a coinbase transaction can be spent and the miner can claim their rewards.⁸

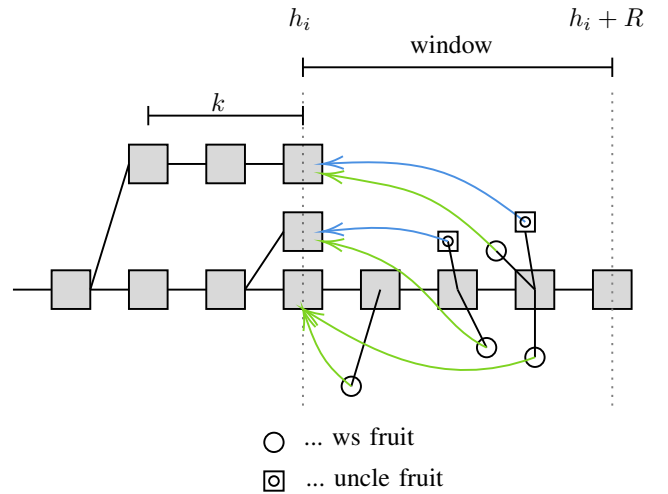


Fig. 1. Uncles can be referenced up to R blocks away. No blocks that are a fork of more than k blocks can be referenced. At every height $h_i + R$, the block rewards at height h_i are distributed, proportional to the work of the valid blocks/uncles at height h_i .

⁷Note that the distribution of mining power is not known in practice, so an approximation will be made based on workshares (cf. Section VI).

⁸<https://learnmeabitcoin.com/technical/mining/coinbase-transaction/#coinbase-maturity>

V. THEORETICAL ANALYSIS

Our analysis closely follows that of FruitChains [6], with some parts taken verbatim. The protocol difference mainly affects the analysis of workshare freshness. In the rest of this section, we prove workobject freshness, consistency, growth, and fairness, ultimately leading to incentive compatibility. We denote the probability that any player mines a block in a round as p , and similarly the probability that a player mines a workshare in a round as p_f , such that $q = \frac{p_f}{p}$.

A. Main Theorem

Theorem 1 is the main theoretical result of our work, which shows that the workshare-based protocol guarantees the useful properties that we aim to achieve, namely consistency, chain growth, and fairness.

Theorem 1 (Security of PoEM²). *For any constant $0 < \delta < 1$, and any p, p_f , let $R = 17$, $k_f = 2qRk$, and $T_0 = 5\frac{k_f}{\delta}$. Then, in $\Gamma_{\text{PoEM}^2}^{p, p_f, R}$ -environments, the Workshare protocol denoted $\Pi_{\text{PoEM}^2}(p, p_f, R)$ satisfies*

- k_f -consistency;
- chain growth rate (T_0, g_0, g_1) where $g_0 = (1 - \delta)(1 - \rho)np_f$, $g_1 = (1 + \delta)np_f$.
- fairness (T_0, δ) .

B. Workobject Freshness

A key property is for any Workobject (or *fruit* to follow [6]) mined by an honest player to stay sufficiently fresh to be incorporated.

Let $\text{WOfreshness}(\text{view}, w, k) = 1$ iff for every honest player i and for every round $r < |\text{view}| - w$, if i mines a Workobject at r in view, then for every honest player j there exists an index pos , such that the Workobject is at pos in the record chain (w.r.t. Nakamoto's protocol) of j at every round $r' \geq r + w$, and pos is at least k blocks away from the end of the chain.

Let

$$\text{wait} = 2\Delta + \frac{2k}{\gamma}$$

Lemma 1. *Let $R = 16$. For any p, p_f , for any $\Gamma_{\text{PoEM}^2}^{p, p_f, R}$ -compliant (A, Z) , there exists a negligible function ϵ such that for any $\kappa \in \mathbb{N}$,*

$$\Pr[\text{view} \leftarrow \text{EXEC}^{\Pi_{\text{PoEM}^2}(p, p_f, R)}(A, Z, \kappa) :$$

$$\text{WOfreshness}(\text{view}, \text{wait}, k) = 1] \geq 1 - \epsilon(\kappa)$$

Proof. Disregard the blockchain consistency, liveness, and chain growth failure events, which only happen with negligible probability. Let $\text{wait} = \text{wait}(k, n, \rho, \Delta)$.

- By *blockchain consistency*, at any point in the execution, whenever an honest player mines a Workobject (i.e., a block, an uncle block or a workshare) f , the block pointed to by that Workobject is at some *fixed* position pos on the blockchain of every honest player, now and at every time in the future.

- Note that if $\text{WORK}(\text{Workobject}) > T_{\text{block}}$, it is not clear at the time of mining, whether it will be a block or an uncle block.
- Honest players try to mine Workobjects that point to the last block in their chain.
- Let ℓ denote the length of the chain of the player that mines f ; by definition $\text{pos} = \ell$
- By the description of the protocol, if the Workobject f is mined at round r' , it gets seen by all honest players by round $r' + \Delta$; additionally, when this happens, all honest players attempt to add f to their chain as long as it remains recent (w.r.t. all honest players).
- By *liveness*, the Workobject gets thus referenced by a block that is in the record chain of all honest players at some position pos that is at least k blocks away from the end of the chain by round

$$r' + \Delta + (1 + \delta)\frac{k}{\gamma} \leq r' + \text{wait} - \Delta$$

- By the upper bound on chain growth, at most

$$(1 + \delta)np(\Delta + \frac{2k}{\gamma})$$

blocks are added in time $\text{wait} - \Delta$. Thus, by round $r' + \text{wait} - \Delta$, no honest player has ever had a chain of length ℓ' , such that

$$\ell' > \ell + (1 + \delta)np(\Delta + \frac{2k}{\gamma})$$

- Thus, by round $r' + \text{wait} - \Delta$, for every such honest player's chain length ℓ' , we have

$$\text{pos} = \ell \geq \ell' - (1 + \delta)np(\Delta + \frac{2k}{\gamma})$$

- By our compliance assumption and by Fact 1 and Fact 2, we have that $\gamma \geq \frac{np}{8}$ and $np\Delta < 1$, thus

$$\text{pos} > \ell' - (1 + \delta) - (1 + \delta)16k \geq \ell' - 16k = \ell' = R\kappa$$

- By *consistency*, all honest players agree that f is found at position pos in their blockchain at any point after $r' + \text{wait} - \Delta$; additionally, by the *consistent length property* all honest players agree that position pos is at least k from the end of the chain by $r' + \text{wait} - \Delta + \Delta = r' + \text{wait}$.
- It remains to show that the block B that is referenced by f , is either a valid block of the chain C of honest users or a valid uncle, as this is the condition for Workobjects to be rewarded. Note that honest parties will reference the last block they see in their chain, which is valid, but not necessarily stable. Should it happen, that the block does not end up in C , it thus has to become an uncle. Suppose there exists another block B' that is competing with B . In any block that is building on top of B' , B is treated as a Workobject (uncle). We have already proven, that Workobjects get incorporated into the chain of every

honest player, thus B will be incorporated into the chain of every honest player. $\square$

We also observe the following fact about $wait$, which says that the expected number of Workobjects mined by all players during $wait + 2$ is upper bounded by k_f .

Fact 3. For any p, p_f , any $\Gamma_{\text{PoEM}^2}^{p, p_f, R}$ -compliant (A, Z) ,

$$(wait + 2) \cdot np_f \leq k_f$$

Proof. Note that by Fact 1 and Fact 2, we have that $\gamma > \frac{np}{8}$ and $np\Delta < 1$, thus

$$(wait+2) \cdot np_f = (2\Delta + 2\frac{k}{\gamma} + 2) \cdot qpn \leq 2q + 2k \cdot 8q + 2 \leq 2qRk = k_f$$

$\square$

C. Some Simplifying Assumptions

Towards proving our main theorem we state some simplifying assumptions that can be made without loss of generality. These assumptions (which all follow from properties of the random oracle H) will prove helpful in our subsequent analysis.

- **WLOG1:** We may without loss of generality assume that honest players never query the RO on the same input—more precisely, we analyze an experiment where if some honest player wants to query it on an “old” input, it re-samples nonce until the input is “new”; since nonce is selected from $\{0, 1\}^\kappa$, this “re-sampling” experiment is identical to the real one with except with negligible probability, thus we can WLOG analyze it.
- **WLOG2:** We may without loss of generality assume that any fruit that points to a block b which was first mined at time t , has been mined after t . Additionally, any fruit that points to a block that comes after b in a valid chain must have been mined after t . (If not, we can predict the outcome of the random oracle H on some input before having queried H which is a contradiction. We omit the standard details.)
- **WLOG3:** We may assume without loss of generality that all fruit mined by honest players are “new” (i.e., different from all valid fruit previously seen by honest players); this follows from WLOG1 and the fact that the probability of seeing a collision in the random oracle is negligible (by a simple union bound over the number of random oracle queries).
- **WLOG4:** We may assume without loss of generality that any valid fruit which appears in some honest players chain at round r was mined before r ; this follows from the unpredictability of the random oracle (and a simple union bound over the number of random oracle queries).
- **WLOG5:** We may assume without loss of generality that there are no “blockchain collisions”—namely, there are no two *different* valid sequences of blocks which end with the same block.

We now turn to proving the three security properties.

D. Workobject Consistency

Disregard chain growth, consistency, and chain quality failure events—they happen with negligible probability. Consider some view $view$ in the support of $EXEC^{\Pi_{\text{PoEM}^2}(p, p_f, R)}(A, Z, \kappa)$, rounds r, r' , s.t. $r' \geq r$, and players i, j that are honest respectively at r, r' in view. By *consistency*, the chains of i, j at r, r' agree except for potentially the last k blocks in the chain of i . Let $C = b_0, \dots, b_{|C|}$ denote those blocks on which they agree and let $b_{|C|+1}, \dots$ denote the maximum k blocks in the chain of i at r which are not in the chain of j at r' . We now bound the number of Workobjects that can be referenced in these remaining (maximum k) inconsistent blocks.

- By the “recency condition” of valid fruit, any valid fruit in the chain of i at r which is after C must point to a block $b_{j'}$ such that $j' > |C| - Rk$.
- By the *chain quality condition*, there exists some j'' s.t. $|C| - Rk - k \leq |C| - Rk$ and $b_{j''}$ was mined by an honest player. Let r'_0 denote the round when this block was mined.
- Note that r'_0 was mined by an honest player holding a chain of length $j'' \geq |C| - Rk - k$; additionally, at r , i is honest, holding a chain of length at most $|C| + k$ (recall that $\text{---}C\text{---}$ contains the blocks on which i and j agree, and by consistency, all but the last k blocks in the chain of i must be in the chain of j). Thus, by the *chain growth upper bound*, at most

$$\mu = (1 + \delta) \frac{2k + Rk}{np}$$

rounds could thus have been elapsed between r'_0 and r .

- By WLOG2, any fruit which gets added after C must have been mined after r'_0 . By WLOG4, any such fruit that is part of the chain of i by r was mined before r .
- We thus conclude by the Chernoff bound that for every sufficiently small δ' except with probability $e^{-\Omega(np_f \cdot \frac{k(R+2)}{np})} = e^{-\Omega(q(R+2)k)}$, there were at most

$$(1 + \delta')^2 \cdot np_f \cdot \frac{k(R+2)}{np} = (1 + \delta')^2 q(R+2)k < 2qRk = k_f$$

“inconsistent” fruits in the chain of i at r

E. Workobject Growth

Consistent length. The consistent length property follows directly from the consistent length property of the underlying blockchain.

Lower bound. Consider any r, t and players i, j that are honest respectively at round r and $r + t$. Consider the t rounds starting from round r .

- By the Workobject freshness condition, every Workobject that is mined by some honest party by round $r + t - wait$ gets incorporated into (and remains in) the chain of player j by $r + t$.

- By the Chernoff bound, in the $t - \text{wait}$ rounds from r to $r + t - \text{wait}$, except with probability $e^{-\Omega((t-\text{wait})\alpha_f)}$, the honest parties mine at least

$$(1 - \delta')(t - \text{wait})\alpha_f$$

Workobjects (where δ' is some arbitrarily small constant), which are all included in the chain of j at $r + t$. Additionally, by WLOG3 they are all “new” (i.e., not included in the chains of i at r) and different.

- Finally, by Workobject consistency (proved in Section 5.4), we have that all but potentially k_f of the Workobjects in the chain of i at r are still in the chain of j at $r + t$.
- We conclude that, except with probability $e^{-\Omega((t-\text{wait})\alpha_f)}$ the chain of j at $r + t$ contains at least

$$(1 - \delta')(t - \text{wait})\alpha_f - k_f$$

more Workobjects than the chain of j at r . By Fact 3, $\text{wait} \cdot \alpha_f = \text{wait} \cdot (1 - \rho)np_f \leq (1 - \rho)k_f \leq k_f$; thus, have at least

$$(1 - \delta)(t - \text{wait})\alpha_f - k_f \geq (1 - \delta)\alpha_f t - 2k_f \quad (2)$$

new Workobject.

We conclude by noting that this implies that a Workobject growth lowerbound of $g_0 = \frac{1}{1+\delta}\alpha_f \geq (1 - \delta)\alpha_f$ in the desired regime: Consider any $T \geq \frac{5k_f}{\delta}$ and any

$$t \geq \frac{T}{g_0} = \frac{T}{\frac{\alpha_f}{1+\delta}}$$

As shown above (see Equation (2)), except with a probability of $e^{-\Omega((t-\text{wait})\alpha_f)}$ the chain must have grown by at least

$$T(1 + \delta)(1 - \delta') - 2k_f = T(1 + \frac{\delta}{2})(1 - \delta') + T\frac{\delta}{2}(1 - \delta') - 2k_f$$

For a sufficiently small δ' the first term is greater than T , and the second term is greater than $2k_f$ and thus the chain must have grown by at least T . Finally note that by Equation (2)

$$e^{-\Omega((t-\text{wait})\alpha_f)} = e^{-\Omega(t\alpha_f - k_f)} = e^{-\Omega(T - k_f)} = e^{-\Omega(5k_f - k_f)} = e^{-\Omega(k)}$$

Thus, the chain growth is guaranteed except with negligible probability.

Upper bound. Disregard the chain growth, consistency and chain quality failure events—they happen with negligible probability. Consider some view view in the support of $\text{EXEC}^{\Pi_{\text{PoEM}^2(p, p_f, R)}}(A, Z, \kappa)$, rounds r, r' , s.t. $r' = r + t$, and players i, j that are honest respectively at r, r' in view. By *consistency*, the chains of i, j at r, r' agree except for potentially the last k blocks in the chain of i . Let $C = b_0, \dots, b_{|C|}$ denote those blocks on which they agree and let $b_{|C|+1}, \dots$ denote the blocks in the chain of j at r' which are not in the chain of i at r (there may be more than k such blocks since we are looking at the chain of j a later time r'); We now upper bound

the number of Workobjects in the new blocks in the chain of j which come after C , similarly to the Workobject consistency proof (the main difference is that we now consider the chain of j as opposed to the chain of i). The details follow:

- By the “recency condition” of valid Workobject, any valid Workobject in the chain of j at r' which is after C must point back to a block $b_{j'}$, such that $j' > |C| - Rk$.
- By the *chain quality condition*, there exists some j'' s.t. $|C| - Rk - k \leq j'' \leq |C| - Rk$ and $b_{j''}$ was mined by an honest player. Let r'_0 denote the round when this block was mined.
- Note that at $r'_0, b_{j''}$ was mined by an honest player holding a chain of length $j'' \geq |C| - Rk - k$; additionally, at r , i is honest, holding a chain of length at most $|C| + k$ (recall that $|C|$ contains the blocks on which i and j agree, and by consistency, all but the last k blocks in the chain of i must be in the chain of j). Thus, by *chain growth upper bound*, for any arbitrarily small δ' at most

$$\mu = (1 + \delta') \frac{2k + Rk}{np}$$

rounds could thus have elapsed between r'_0 and r .

- By WLOG2, any Workobject which gets added after C must have been mined after r'_0 . By WLOG4, any such Workobject that is part of the chain of j by r' was mined before r' .
- We thus conclude by the Chernoff bound that except with a probability of $e^{-\Omega(np_f \cdot \frac{k(R+2)}{np})} = e^{-\Omega(q(R+2)k)}$, there were at most

$$(1 + \delta')^2 \cdot np_f \cdot \left(\frac{k(R+2)}{np} + t \right) = (1 + \delta')^2 (q(R+2)k + np_f t) \leq k_f + (1 + \delta')^2 np_f t \quad (3)$$

“new” Workobjects in the chain of j at r' .

We conclude by noting that this implies a Workobject growth upper bound of $g_1 = (1 + \delta)np_f$ in the desired regime: Consider any $T \geq \frac{5k_f}{\delta}$ and any

$$t = \frac{T}{g_1} = \frac{T(1 + \delta)}{np_f}$$

As shown above (see Equation (3)), during this time t , except with negligible probability, the chain must have grown by at most

$$k_f + (1 + \delta')^2 T(1 - \delta) \leq T\delta/5 + (1 + \delta')^2 T/(1 + \delta)$$

For any $0 < \delta < 1$ and $\delta' = 0.1\delta$, the above expression is upper bounded by T .

F. Workobject Fairness

Disregard the chain growth, chain quality, Workobject freshness, and Workobject growth failure events, which happen with negligible probability. Consider some ϕ -fraction player subset section S , some view view in the support of

$EXEC^{\Pi_{\text{PoEM}^2(p,p_f,R)}}(A, Z, \kappa)$, some round r and some player i that is honest in round r of view. Let $C = b_0, \dots, b_{|C|}$ be the blocks in the view of i at r , let $f_0, \dots, f_\ell$ be the Workobjects contained in them, and let $m_0, \dots, m_\ell$ be the records contained in the Workobjects; let $f_j, \dots, f_{j+T}$ be the T consecutive Workobjects for some j , where $T \geq \frac{5k_f}{\delta}$.

Let r_0 be the round when the *block* in the view of i at r containing f_{j+k_f} was first added to *some* honest player j_0 's chain; let r_1 be the round when the block (again in the view of i at r) containing f_{j+T} was first added to some honest player j_1 's chain, and let $t = r_1 - r_0 - 2$ be the number of rounds from $r_0 + 1$ to $r_1 - 1$. We lower bound the number of S -compatible (honest) Workobjects in the sequence, similar to the proof of Workobject growth *lower bound*:

- By the *Workobject freshness* condition, every Workobject mined by some honest player between $(r_0 + 1)$ and $(r_1 - 1) - \text{wait}$ will be in the chain of j_1 at some position pos that is at least k positions from the end of the chain, before the beginning of round r_1 and will remain so.
- By the Chernoff bound, in the $t - \text{wait}$ rounds from $r_0 + 1$ to $(r_1 - 1) - \text{wait}$, except with probability $e^{-\Omega((t - \text{wait})\phi np_f)}$, the honest parties in S mine at least $(1 - \delta')(t - \text{wait})\phi np_f$ Workobjects (where δ' is some arbitrarily small constant), which thus are all included in the chain of j_1 by $r_1 - 1$.
- Since Workobjects are ordered by the block containing them, and since in round r_1 a *new* block is added which contains f_{j+T} , it follows from *blockchain consistency* that all these Workobjects contained in the sequence $f_1, \dots, f_{j+T}$ (recall that all these Workobjects are found in blocks that are at least k positions from the end of the chain, so by consistency, those blocks cannot change and thus were not added in round r_1 and consequently must come before the block containing f_{j+T}).
- By WLOG3, these Workobjects are also all “new” (i.e., not included in the chains of j_0 at r_0) and different. Since in round r_0 , the block containing f_{j+k_f} was added to the chain of j_0 , and since by WLOG5, the chain of j_0 at r_0 up until (and including) the block which contains f_{j+k_f} is a prefix of C , all these Workobjects must in fact be contained in the sequence $f_{j+k_f}, \dots, f_{j+T}$.
- Finally, by Workobject consistency, at r_0 all honest players' Workobject chains contain $f_1, \dots, f_j$ (since recall that some player added f_{j+k_f} at r_0). Thus, all these Workobjects are S -compatible w.r.t. the prefix $f_1, \dots, f_{j-1}$ before the T segment we are considering.

We proceed to show that t is sufficiently large. Recall that j_0 is honest at r_0 and j_1 is honest at r_1 . We know that at r_1 , the Workobject chain contains at least f_{j+T} Workobjects. Additionally, at r_0 the Workobject f_{j+k_f} is added for the first time, so by Workobject consistency, at most $j + 2k_f$ Workobjects could have been in the chain of i at this point (since a Workobject at position $j + k_f$ is modified). Thus, the Workobject chain must have grown by at least $T - 2k_f$ from r_0 to r_1 . By the *upper bound on Workobject growth*

we have that: $T - 2k_f \leq k_f + (1 - \delta')^2 np_f(t + 2)$. Thus: $t \geq \frac{1}{(1 + \delta')^2 np_f}(T - 3k_f) - 2$.

We conclude that (except with negligible probability) the number of Workobjects in the sequence is at least:

$$\begin{aligned} (1 - \delta')\phi np_f \left(\frac{1}{(1 + \delta')^2 np_f} (T - 3k_f) - 2 - \text{wait} \right) &= \\ (1 - \delta')\phi \left(\frac{1}{(1 + \delta')^2} (T - 3k_f) - np_f(\text{wait} + 2) \right) &\geq \\ (1 - \delta')\phi \left(\frac{1}{(1 + \delta')^2} (T - 3k_f) - k_f \right) &\geq \\ (1 - \delta')\phi \left(\frac{1}{(1 + \delta')^2} (T - 4.5k_f) \right) &\geq \\ \phi(T - 5k_f) \end{aligned}$$

G. Incentive Compatibility

Our final theoretical result is game theoretic and concerns utility-maximizing participants.

Any secure blockchain protocol that satisfies δ -approximate fairness (where $\delta < 0.3$) w.r.t $T(\kappa)$ length windows can be used as the ledger underlying a cryptocurrency system while ensuring 3δ -incentive compatibility if players (i.e. miners) only care about how much money they receive.

We say that honest mining is a ρ -coalition-safe ϵ -Nash equilibrium if, with overwhelming probability, no $\rho' < \rho$ fraction coalition can gain more than a multiplicative factor $(1 + \epsilon)$ in utility, no matter what transactions are being processed—formally, consider some environment providing transactions into the system. We restrict to a setting where the total rewards and transaction fees during the run of the system is some fixed constant V .

Fairness implies that no matter what deviation the coalition performs, with overwhelming probability, the fraction of adversarial blocks in any $T(\kappa)$ - length window of the chain is upper bounded by $(1 + \delta)\rho$ and thus the total amount of compensation received by the attacker is bounded by $(1 + \delta)\rho \cdot V$; in contrast, by fairness, if the coalition had been following the honest protocol, they are guaranteed to receive at least $(1 - \delta)\rho \cdot V$; thus, the multiplicative increase in utility is: $\frac{1 + \delta}{1 - \delta} \leq 1 + 3\delta$ when $\delta < 0.3$.

VI. EVALUATION

We now compare the Proportional Reward Splitting (PRS) mechanism to other reward mechanisms under realistic parameters. In order to perform the comparison we utilized the Markov Decision Process (MDP) implementation of [7]. MDPs are fitting tools in our case since we want to evaluate the optimal strategies w.r.t. a specific utility. Therefore, when parameterizing our MDPs with realistic values, e.g., in terms of block finality, we can identify the optimal strategy that utility-maximizing rational parties would opt for.

The model does not require knowledge of the identities of parties, or which ones are adversarial, but only assumes that the adversary A holds a certain amount of power. In practice, a protocol's designer could use our analysis to get a lower bound

on their protocol’s performance, by pinpointing the advantage of the strongest adversary that their protocol can tolerate.

Following, we first outline the construction of the MDP and then compare PRS with other reward mechanisms under three metrics, namely incentive compatibility, subversion gain, and censorship susceptibility. Following, we evaluate how different values of the PRS’s parameters, namely the workshare and fork eligibility windows, affect its performance. Finally, we analyze the accuracy of estimating the adversarial and honest parties’ power via workshares and outline the tradeoff between sampling accuracy and storage overhead.

A. Description of Markov Decision Process

The MDP implementation of PRS is a direct adaptation of the vanilla Reward Splitting (RS) MDP. Here we give a brief description of the MDP and refer to [7] for more details.

The state of the MDP is a tuple of four elements $\langle l_a, l_c, \text{fork}, \text{history} \rangle$:

- l_a denotes the length of the (private) adversarial chain;
- l_c denotes the length of the honest (public) chain;
- fork takes three values: *active* if there is a tie, i.e., honest miners are split between two chains; *cLast* if the latest block is honest; *aLast* if the latest block is adversarial.
- history is a bitstring as follows: (i) its length represents the number of consecutive attacker blocks in the common (main) chain; (ii) each position corresponds to a height of the main chain, starting from the last block (the tip of the chain); (iii) a 0 bit denotes that no competing honest block exist for the corresponding height, whereas 1 means that an honest block for that height exists.

State transitions occur via the following actions:

- *Adopt*: A adopts the public chain.
- *Wait*: A keeps mining privately and publish no blocks.
- *Match*: A publishes a chain of private blocks equal to the length of the public chain, causing a tie; this is feasible when $l_a \geq l_c$ and fork = *cLast*.
- *Override_k*: A publishes a chain of private blocks which is k blocks longer than the public chain.

Finally, the reward allocation is the point of difference between PRS and RS. In RS, when choosing *Adopt* the honest miners and A each receive half of the rewards for heights where block honest and adversarial blocks exist. Instead, in this case in PRS the rewards are split proportionally to each side’s mining power; for instance, if the honest miners (are set to) have 70% of mining power, they receive 70% of the rewards for these heights, instead of 50% as in RS.⁹ For the heights where only honest blocks exist, the honest miners get the full reward. Additionally, the adversary receives the full reward for heights where honest objects (blocks and/or workshares) are “pushed out” of the history, i.e., the adversary creates enough consecutive blocks to censor their inclusion.

⁹In practice it is impossible to know the exact power distribution between honest parties and the adversary. Instead, an approximation is made using workshares. The accuracy of this approximation is explored in Section VI-D.

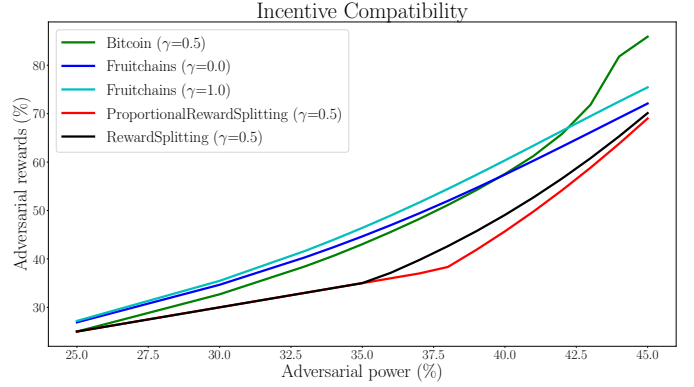


Fig. 2. Incentive compatibility for various reward mechanisms. The eligibility window for fruits (in FruitChains) and objects (in Proportional Reward Splitting) is set to $w = 6$. The fork eligibility window for FruitChains, Reward Splitting, and Proportional Reward Splitting is set to $k = 6$. Larger values indicate worse performance.

B. Comparison of Reward Mechanisms

We first explore the metrics established in [7], namely: (i) incentive compatibility; (ii) subversion gain; (iii) censorship susceptibility. For each metric, we compare PRS with Bitcoin, FruitChains, and vanilla RS.

In all evaluations we used the same parameters for the eligibility windows. The first window concerns object eligibility, that is the number of block heights within which an object should be published in order to be eligible for rewards. This window was set to $w = 6$. The second window concerns fork eligibility, that is the depth of a fork up to which objects (that point to blocks of that depth) are eligible for rewards. This window was set to $k = 6$.

The parameter γ , which indicates the percentage of honest miners that adopt the adversarial forks, is set to 0.5 for Bitcoin, RS, and PRS, and 0 and 1 for FruitChains.¹⁰ In essence, when two chains of the same height are available, 0.5 implies a random choice, which — relevant to our setting — is almost equivalent to the PoEM fork choice rule.

Finally, in FruitChains we set the block-to-fruit ratio to 1.

Incentive Compatibility

The first metric of comparison is incentive compatibility (IC). Briefly, IC expresses the percentage of rewards that the adversary gains compared to its fair share, that is its relative mining power. Figure 2 depicts the IC for the reward mechanisms under question. In essence, values above the line $x = y$ indicate that the adversary gains disproportionately more rewards, compared to its mining power, so larger values indicate worse IC performance.

As can be seen, PRS is better than all other mechanisms. In particular, for adversarial mining power 25% all mechanisms, except FruitChains, offer optimal IC. However, as the adver-

¹⁰For FruitChains we use both 0 and 1 because the FruitChains MDP implementation of [7] did not enable using 0.5 due to complexity issues.

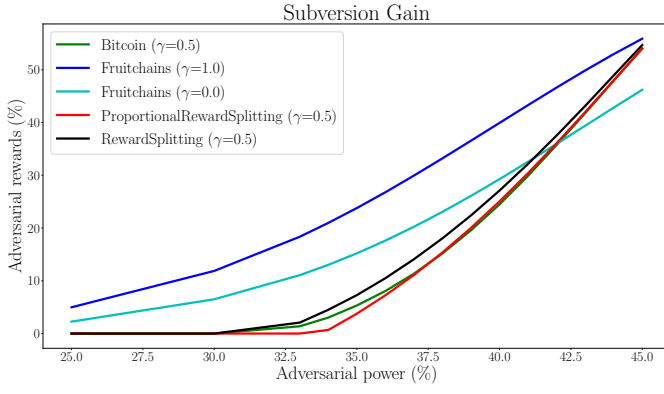


Fig. 3. Comparison of subversion gain for various reward mechanisms. The eligibility window for fruits (in FruitChains) and objects (in Proportional Reward Splitting) is set to $w = 6$. The fork eligibility window for FruitChains, Reward Splitting, and Proportional Reward Splitting is set to $k = 6$. Larger values indicate worse performance.

arial power increases, RS and PRS perform optimally up to 35%, after which point PRS outperforms RS.

Subversion Gain

Subversion gain indicates the profitability of double spending attacks. In particular, the profit from a double spending attack is time averaged and set to 3 times the block reward. In other words, if the adversary manages to orphan k blocks during the attack, it is granted $3 \cdot k$ rewards. The confirmation time is set to 6 blocks, so the attack is successful if the adversary reverts 6 blocks in order to successfully double spend the assets. If the attack is unsuccessful, the adversary gets no rewards, but also incurs no penalty.

Figure 3 depicts the comparison of the reward mechanisms w.r.t. subversion gain. For the most part, PRS is again the better option. In particular, up to 30% all mechanisms except FruitChains perform optimally, i.e., the adversary has zero gain. For the range of adversarial power between 30% and 38% PRS is the best option, whereas between 38% and 42% it is in effect equally good to Bitcoin. Above 42%, FruitChains with $\gamma = 0$ performs better, which is a result of the particularly favorable setting (compared to $\gamma = 0.5$ for the other mechanisms); instead, under $\gamma = 1$, FruitChains is strictly worse than all others for all adversarial powers.

In summary, PRS is optimal w.r.t. subversion gain and, in some cases, is strictly better than all other options.

Censorship Susceptibility

Censorship susceptibility indicates the maximum fraction of income loss that the adversary can incur on honest miners under the threat of a censorship attack. Specifically, A 's reward corresponds to the honest miners' loss in the form of orphaned blocks.

Figure 4 depicts the results for censorship susceptibility. These are consistent with [7], wherein FruitChains outperforms the other mechanisms for smaller adversaries but becomes relatively worse for larger values. Interestingly, PRS

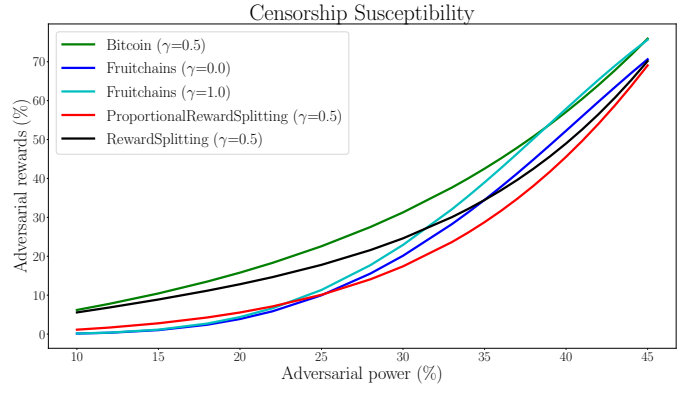


Fig. 4. Comparison of censorship susceptibility for various reward mechanisms. The eligibility window for fruits (in FruitChains) and objects (in Proportional Reward Splitting) is set to $w = 6$. The fork eligibility window for FruitChains, Reward Splitting, and Proportional Reward Splitting is set to $k = 6$. Larger values indicate worse performance.

strictly outperforms vanilla RS for all adversarial powers. As a result, FruitChains is the best option only for adversarial power up to 25%, after which point PRS is the best option. Notably, even for smaller values of adversarial power, the difference between PRS and FruitChains is less than 2%.

C. Comparison of Eligibility Parameters

We now evaluate how the different eligibility parameters affect the performance of the reward mechanisms. In particular, we will compare the two types of reward splitting w.r.t. the fork and the workshare eligibility windows. The metric of comparison is incentive compatibility, since all metrics are affected in the same manner when changing the parameters. As before, we set γ to be 0.5.

Workshare Eligibility Window: The first parameter is the workshare eligibility window w . This is the distance between the block that a workshare references and the block within which it gets published. Intuitively, larger window values require the adversary to censor the chain for longer periods of time, in order to force an honest workshare to become ineligible, thus making the attack harder. However, they also incur a delay on the finalization of objects and, consequently, the allocation of rewards.

Figure 5 depicts how the two reward splitting mechanisms perform for three workshare eligibility window values, $w \in [3, 6, 9]$. Evidently, Proportional Reward Splitting outperforms vanilla Reward Splitting in all cases. As expected, larger values of w result in better performance, that is lower adversarial rewards. Consequently, PRS is optimal for larger values of adversarial power; when $w = 3$, PRS is optimal for adversarial power up to 33%, whereas for $w = 9$ this goes up to 38%.

Fork Eligibility Window: The second parameter is the fork eligibility window k . This window sets the depth of a fork up to which objects are eligible for rewards. Specifically, if $k = 3$, then objects that point to a block in a fork are eligible for rewards only if this block is among the first 3 of the fork. Note that the height of the workshare, which is considered for

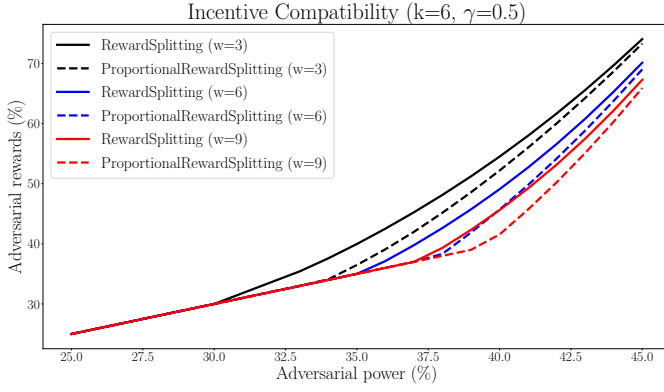


Fig. 5. Incentive compatibility w.r.t. the workshare eligibility window between Reward Splitting and Proportional Reward Splitting. The fork window for objects is set to $k = 6$. Larger values indicate worse performance.

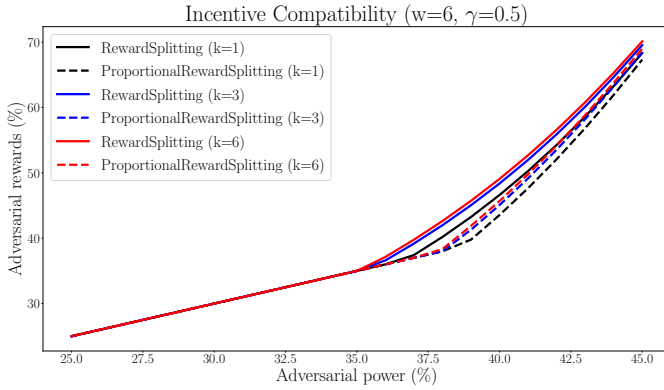


Fig. 6. Incentive compatibility for various values of the fork window between Reward Splitting and Proportional Reward Splitting. The eligibility window for objects is set to $w = 6$. Larger values indicate worse performance.

the workshare eligibility window, is computed as the height of the referenced block. For example, if a workshare points to the second block of a fork which occurred at height 100, then the workshare's height is 102 and, in order to be eligible for rewards, it should be published in a (main chain) block up to height $102 + w$.

Figure 6 depicts the effect of the fork eligibility window on incentive compatibility. Evidently, lower values of k result in better performance, i.e., lower adversarial rewards. One explanation for this could be that lower values of k penalize forking more, so the adversary is incentivized to abandon forks more often, hence its rewards get closer to its fair share.

Interestingly, we observe that PRS outperforms RS even for different values of k . Specifically, PRS under $k = 6$ performs better (for the most part) than RS under $k = 1$.

D. Sampling Accuracy

So far our evaluation of the proportional reward splitting mechanism assumes a perfect knowledge of the distribution of power between the honest parties and the adversary. In practice this is not possible to achieve for Proof-of-Work

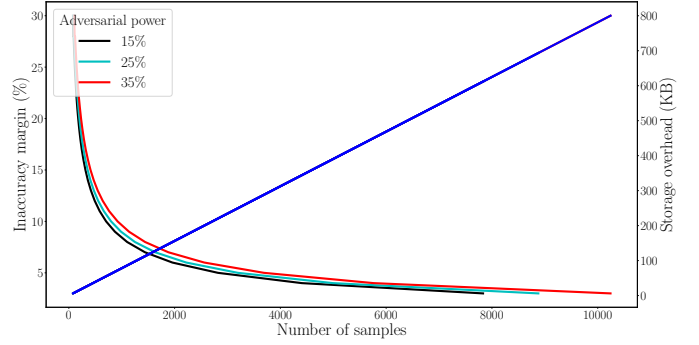


Fig. 7. Evaluation of the number of workshares (samples) needed to estimate the distribution of power between honest parties and the adversary. In all cases, the error bound of the observation is set to $\epsilon = 0.05$. Inaccuracy corresponds to the acceptable deviation of the observation from the correct (ideal) value. Storage overhead corresponds to the size of the needed samples, where each sample is 80 bytes.

(PoW) ledgers, as each party's proportional power is not visible to anyone, not even to the party itself. Although a party knows their *absolute* power, they do not know the exact amount of power of other parties.

Instead, the distribution of power can be inferred by observed events, specifically block creations. Since the probability that each party creates a block on each round is proportional to their power and each such event is independent, a large enough sample of blocks enables inferring power distribution.

A pertinent question is how many samples are needed to make an accurate enough estimation, with a low margin of error. In our setting, the samples correspond to workshares and blocks. Therefore, the question of interest is how many workshares should be published per height, such that each block's rewards are distributed proportionally to each side's power, accurately and on most cases (low margin of error).

To estimate the number of workshares needed, we model the generation of workshares as a series of independent Bernoulli trials. Let $X_i \in \{0, 1\}$ be the i -th sample of random variable X , where 0 corresponds to an adversarial workshare creation and 1 corresponds to honest workshare. The probability of 1 for each X_i is p , so $\{X_i\}_{i \in [n]}$ is independent and identically distributed to a Bernoulli trial $\text{Bern}(p)$.

Let $X = \sum_{i=1}^n X_i$ be the random variable that represents the sum of the observed values, i.e., the number of honest workshares. It holds that $E[X] = n \cdot p$, where n is the number of samples (i.e., Bernoulli trials) and p is the probability of creating an honest workshare.

By the Chernoff bound: $\Pr[X \leq (1 + \delta) \cdot E[x]] < \epsilon = e^{-\frac{\delta^2 \cdot n \cdot p}{2}}$, where: (i) δ denotes the acceptable deviation from the completely accurate (expected) value; (ii) ϵ denotes the error bound of the observation, i.e., the probability that the observed value is less accurate than δ . By solving for n we find, for a given (amount of honest power) p , the minimum number of workshares needed to get an accurate enough observation with low error probability: $n = -\frac{2 \cdot \log \epsilon}{\delta^2 \cdot p}$.

Figure 7 depicts how n is affected given various values of

the other parameters. Here, we set the error bound to 0.05; in other words, the probability that the observed distribution between the honest and adversarial parties is less accurate than the acceptable level of inaccuracy is 5%.

The x axis corresponds to the number of workshares, while the left y axis corresponds to the acceptable level of inaccuracy. We observe that inaccuracy drops exponentially w.r.t. the number of samples used. Additionally, inaccuracy drops at a faster rate when the adversarial power is lower. The right y axis corresponds to the storage overhead from publishing the relevant number of workshares on-chain. We assume that each workshare is equivalent to a Bitcoin block's header, which amounts to 80 bytes. Therefore, the overhead *per block* increases linearly to the number of workshares.

For example, assume adversarial power is 15%. 7,832 workshares achieve 3% accuracy (with error probability 5%), requiring 612 KB per block. Similarly, 1,000 workshares achieve approx. 8.5% accuracy at 79 KB. On the other hand, assuming 35% adversarial power, 3% accuracy is achieved with 10,242 workshares, which corresponds to 800 KB.

VII. CONCLUSION

In this work we introduce Proportional Reward Splitting (PRS), a reward mechanism for blockchain-based distributed ledgers. Its core idea is using workshares, which are block-like objects with lower amounts of Proof-of-Work (PoW), to estimate the mining power distribution among parties, and then allocate a fixed reward per height among all participants proportionally to their power. Our analysis shows that, for large enough parameters, PRS is an approximate Nash equilibrium that guarantees fairness. Additionally, we show that, for realistic and practical parameter values, PRS outperforms the state-of-the-art reward allocation mechanisms across a range of metrics. Finally, we evaluate the tradeoff between accuracy of the power distribution estimation and storage overhead, showing that, for realistic adversaries, accuracy within 3% can be achieved with a few hundred KB overhead per block.

Our work also leaves various questions open. Although PRS improves the state-of-the-art, it is still not optimal in practice. Therefore, the question remains whether a reward mechanism can be proven game theoretically secure for practical parameter values, instead of unrealistically large ones. Additionally, PRS could be considered alongside DAG-based ledgers, exploring if and how this combination could be more performant. Finally, although we considered PRS in a PoW context, where the exact distribution of power among participants is unknown and needs to be estimated, the ideas of PRS could be used in the context of Proof-of-Stake, where power is recorded on-chain and is known at the protocol level.

REFERENCES

- [1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.
- [2] I. Eyal and E. G. Sirer, "Majority is not enough: bitcoin mining is vulnerable," *Commun. ACM*, vol. 61, no. 7, pp. 95–102, 2018. [Online]. Available: <https://doi.org/10.1145/3212998>
- [3] A. Sapirshstein, Y. Sompolinsky, and A. Zohar, "Optimal selfish mining strategies in bitcoin," in *Financial Cryptography and Data Security - 20th International Conference, FC 2016, Christ Church, Barbados, February 22-26, 2016, Revised Selected Papers*, ser. Lecture Notes in Computer Science, J. Grossklags and B. Preneel, Eds., vol. 9603. Springer, 2016, pp. 515–532. [Online]. Available: https://doi.org/10.1007/978-3-662-54970-4_30
- [4] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun, "On the security and performance of proof of work blockchains," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, Eds. ACM, 2016, pp. 3–16. [Online]. Available: <https://doi.org/10.1145/2976749.2978341>
- [5] K. Nayak, S. Kumar, A. Miller, and E. Shi, "Stubborn mining: Generalizing selfish mining and combining with an eclipse attack," in *IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21-24, 2016*. IEEE, 2016, pp. 305–320. [Online]. Available: <https://doi.org/10.1109/EuroSP.2016.32>
- [6] R. Pass and E. Shi, "Fruitchains: A fair blockchain," in *Proceedings of the ACM Symposium on Principles of Distributed Computing, PODC 2017, Washington, DC, USA, July 25-27, 2017*, E. M. Schiller and A. A. Schwarzmann, Eds. ACM, 2017, pp. 315–324. [Online]. Available: <https://doi.org/10.1145/3087801.3087809>
- [7] R. Zhang and B. Preneel, "Lay down the common metrics: Evaluating proof-of-work consensus protocols' security," in *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 175–192. [Online]. Available: <https://doi.org/10.1109/SP.2019.00086>
- [8] S. D. Lerner, "Decor+ hop: A scalable blockchain protocol," *Semantic Scholar*, 2015.
- [9] P. Camacho and S. D. Lerner, "Decor+ lami: A scalable blockchain protocol," in *Scaling Bitcoin Workshops*, 2016.
- [10] P. R. Rizun, "Subchains: A technique to scale bitcoin and improve the user experience," *Ledger*, vol. 1, pp. 38–52, 2016. [Online]. Available: <https://ledgerjournal.org/ojs/index.php/ledger/article/view/40>
- [11] K. Kreder, S. Shastri, A. Tzinas, S. Vishwanath, and D. Zindros, "A better proof-of-work fork choice rule," *Cryptology ePrint Archive, Paper 2024/200*, 2024, <https://eprint.iacr.org/2024/200>. [Online]. Available: <https://eprint.iacr.org/2024/200>
- [12] P. Szalachowski, D. Reijnders, I. Homoliak, and S. Sun, "Strongchain: Transparent and collaborative proof-of-work consensus," in *28th USENIX Security Symposium, USENIX Security 2019, Santa Clara, CA, USA, August 14-16, 2019*, N. Heninger and P. Traynor, Eds. USENIX Association, 2019, pp. 819–836. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/szalachowski>
- [13] A. Zamyatin, N. Stifter, P. Schindler, E. R. Weippl, and W. J. Knottenbelt, "Flux: Revisiting near blocks for proof-of-work blockchains," *IACR Cryptol. ePrint Arch.*, p. 415, 2018. [Online]. Available: <https://eprint.iacr.org/2018/415>
- [14] I. Eyal, A. E. Gencer, E. G. Sirer, and R. van Renesse, "Bitcoin-ng: A scalable blockchain protocol," in *13th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2016, Santa Clara, CA, USA, March 16-18, 2016*, K. J. Argyraki and R. Isaacs, Eds. USENIX Association, 2016, pp. 45–59. [Online]. Available: <https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eyal>
- [15] E. Kokoris-Kogias, P. Jovanovic, N. Gailly, I. Khoffi, L. Gasser, and B. Ford, "Enhancing bitcoin security and performance with strong consistency via collective signing," in *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*, T. Holz and S. Savage, Eds. USENIX Association, 2016, pp. 279–296. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/kogias>
- [16] R. Zhang and B. Preneel, "Publish or perish: A backward-compatible defense against selfish mining in bitcoin," in *Topics in Cryptology - CT-RSA 2017 - The Cryptographers' Track at the RSA Conference 2017, San Francisco, CA, USA, February 14-17, 2017, Proceedings*, ser. Lecture Notes in Computer Science, H. Handschuh, Ed., vol. 10159. Springer, 2017, pp. 277–292. [Online]. Available: https://doi.org/10.1007/978-3-319-52153-4_16
- [17] J. A. Garay, A. Kiayias, and N. Leonardos, "The bitcoin backbone protocol: Analysis and applications," in *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia,*

Bulgaria, April 26-30, 2015, *Proceedings, Part II*, ser. Lecture Notes in Computer Science, E. Oswald and M. Fischlin, Eds., vol. 9057. Springer, 2015, pp. 281–310. [Online]. Available: https://doi.org/10.1007/978-3-662-46803-6_10

- [18] S. Azouvi and A. Hicks, “Sok: Tools for game theoretic models of security for cryptocurrencies,” *CoRR*, vol. abs/1905.08595, 2019. [Online]. Available: <http://arxiv.org/abs/1905.08595>
- [19] R. Pass, L. Seeman, and A. Shelat, “Analysis of the blockchain protocol in asynchronous networks,” in *Advances in Cryptology – EUROCRYPT 2017*, J.-S. Coron and J. B. Nielsen, Eds. Cham: Springer International Publishing, 2017, pp. 643–673.